

Small. Fast. Reliable. Choose Any Three.

Using

SQLite



O'REILLY®

Jay A. Kreibich

What Is SQLite?

In the simplest terms, SQLite is a public-domain software package that provides a *relational database management system*, or RDBMS. Relational database systems are used to store user-defined records in large tables. In addition to data storage and management, a database engine can process complex query commands that combine data from multiple tables to generate reports and data summaries. Other popular RDBMS products include Oracle Database, IBM's DB2, and Microsoft's SQL Server on the commercial side, with MySQL and PostgreSQL being popular open source products.

The “Lite” in SQLite does not refer to its capabilities. Rather, SQLite is lightweight when it comes to setup complexity, administrative overhead, and resource usage. SQLite is defined by the following features:

Serverless

SQLite does not require a separate server process or system to operate. The SQLite library accesses its storage files directly.

Zero Configuration

No server means no setup. Creating an SQLite database instance is as easy as opening a file.

Cross-Platform

The entire database instance resides in a single cross-platform file, requiring no administration.

Self-Contained

A single library contains the entire database system, which integrates directly into a host application.

Small Runtime Footprint

The default build is less than a megabyte of code and requires only a few megabytes of memory. With some adjustments, both the library size and memory use can be significantly reduced.

Transactional

SQLite transactions are fully ACID-compliant, allowing safe access from multiple processes or threads.

Full-Featured

SQLite supports most of the query language features found in the SQL92 (SQL2) standard.

Highly Reliable

The SQLite development team takes code testing and verification very seriously.

Overall, SQLite provides a very functional and flexible relational database environment that consumes minimal resources and creates minimal hassle for developers and users.

Self-Contained, No Server Required

Unlike most RDBMS products, SQLite does not have a client/server architecture. Most large-scale database systems have a large server package that makes up the database engine. The database server often consists of multiple processes that work in concert to manage client connections, file I/O, caches, query optimization, and query processing. A database instance typically consists of a large number of files organized into one or more directory trees on the server filesystem. In order to access the database, all of the files must be present and correct. This can make it somewhat difficult to move or reliably back up a database instance.

All of these components require resources and support from the host computer. Best practices also dictate that the host system be configured with dedicated service-user accounts, startup scripts, and dedicated storage, making the database server a very intrusive piece of software. For this reason, and for performance concerns, it is customary to dedicate a host computer system solely for the database server software.

To access the database, client software libraries are typically provided by the database vendor. These libraries must be integrated into any client application that wishes to access the database server. These client libraries provide APIs to find and connect to the database server, as well as set up and execute database queries and commands. [Figure 1-1](#) shows how everything fits together in a typical client/server RDBMS.

In contrast, SQLite has no separate server. The entire database engine is integrated into whatever application needs to access a database. The only shared resource among applications is the single database file as it sits on disk. If you need to move or back up the database, you can simply copy the file. [Figure 1-2](#) shows the SQLite infrastructure.

By eliminating the server, a significant amount of complexity is removed. This simplifies the software components and nearly eliminates the need for advanced operating system support. Unlike a traditional RDBMS server that requires advanced multitasking and high-performance inter-process communication, SQLite requires little more than the ability to read and write to some type of storage.

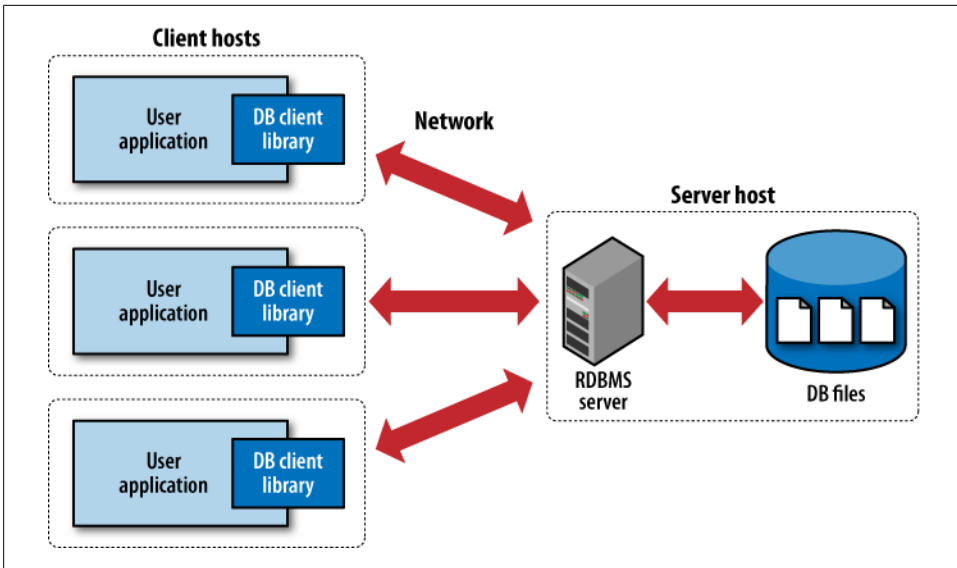


Figure 1-1. Traditional RDBMS client/server architecture that utilizes a client library.

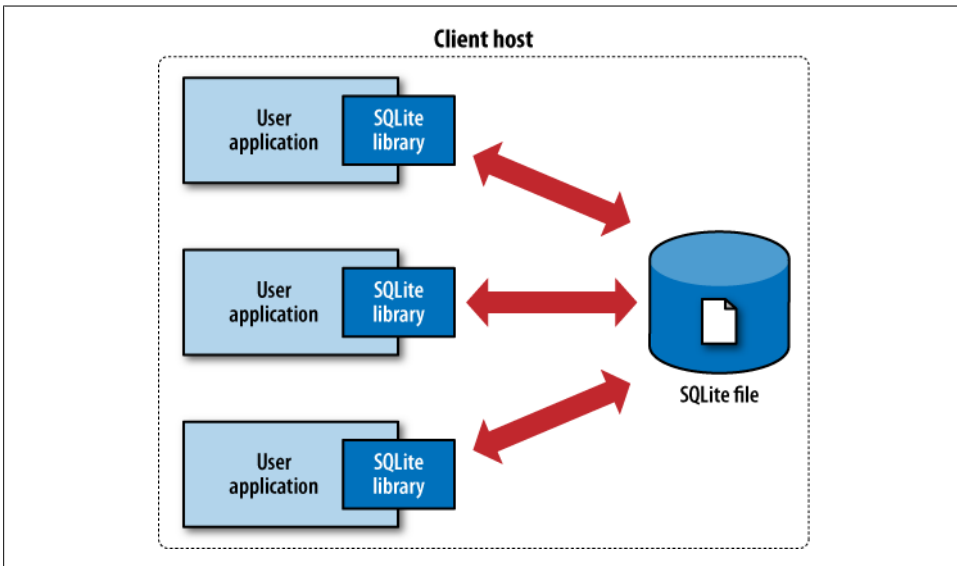


Figure 1-2. The SQLite server-less architecture.

This simplicity makes it fairly straightforward to port SQLite to just about any environment, including mobile phones, handheld media players, game consoles, and other devices, where no traditional database system could ever dare to venture.



Although SQLite does not use a traditional client/server architecture, it is common to speak of applications being “SQLite clients.” This terminology is often used to describe independent applications that simultaneously access a shared SQLite database file, and is not meant to imply that there is a separate server.

SQLite is designed to be integrated directly into an executable. This eliminates the need for an external library and simplifies distribution and installation. Removing external dependencies also removes most versioning issues. If the SQLite code is built right into your application, you never have to worry about linking to the correct version of a client library, or that the client library is version-compatible with the database server.

Eliminating the server imposes some restrictions, however. SQLite is designed to address localized storage needs, such as a web server accessing a local database. This means it isn’t well suited for situations where multiple client machines need to access a centralized database. That situation is more representative of a client/server architecture, and is better serviced by a database system that uses the same architecture.

Single File Database

SQLite packages the entire database into a single file. That single file contains the database layout as well as the actual data held in all the different tables and indexes. The file format is cross-platform and can be accessed on any machine, regardless of native byte order or word size.

Having the whole database in a single file makes it trivial to create, copy, or back up the on-disk database image. Whole databases can be emailed to colleagues, posted to a web forum, or checked into a revision control system. Entire databases can be moved, modified, and shared with the same ease as a word-processing document or spreadsheet file. There is no chance of a database becoming corrupt or unavailable because one of a dozen files was accidentally moved or renamed.

Perhaps most importantly, computer users have grown to expect that a document, project, or other “unit of application data” is stored as a single file. Having the whole database in a single file allows applications to use database instances as documents, data stores, or preference data, without contradicting customer expectations.

Zero Configuration

From an end-user standpoint, SQLite requires nothing to install, nothing to configure, and nothing to worry about. While there are a fair number of tuning parameters available to developers, these are normally hidden from the end-user. By eliminating the server and merging the database engine directly into your application, your customers never need to know they’re using a database. It is quite practical to design an application

so that selecting a file is the only customer interaction—an action they are already comfortable doing.

Embedded Device Support

SQLite's small code size and conservative resource use makes it well suited for embedded systems running limited operating systems. The ANSI C source code tends toward an older, more conservative style that should be accepted by even the most eccentric embedded processor compiler. Using the default configuration, the compiled SQLite library is less than 700 KB on most platforms, and requires less than 4 MB of memory to operate. By omitting the more advanced features, the library can often be trimmed to 300 KB or less. With minor configuration changes, the library can be made to function on less than 256 KB of memory, making its total footprint not much more than half a megabyte, plus data storage.

SQLite expects only minimal support from its host environment and is written in a very modular way. The internal memory allocator can be easily modified or swapped out, while all file and storage access is done through a *Virtual File System* (VFS) interface that can be modified to meet the needs and requirements of different platforms. In general, SQLite can be made to run on almost anything with a 32-bit processor.

Unique Features

SQLite offers several features not found in many other database systems. The most notable difference is that SQLite uses a dynamic-type system for tables. The SQLite engine will allow you to put any value into nearly any column, regardless of type. This is a major departure from traditional database systems, which tend to be statically typed. In many ways, the dynamic-type system in SQLite is similar to those found in popular scripting languages, which often have a single scalar type that can accept anything from integers to strings. In my own experience, the dynamic-type system has solved many more problems than it has caused.

Another useful feature is the ability to manipulate more than one database at a time. SQLite allows a single database connection to associate itself with multiple database files simultaneously. This allows SQLite to process SQL statements that bridge across multiple databases. This makes it trivial to join tables from different databases with a single query, or bulk copy data with a single command.

SQLite also has the ability to create fully in-memory databases. These are essentially database “files” that have no backing store and spend their entire lifetime within the file cache. While in-memory databases lack durability and do not provide full transaction support, they are very fast (assuming you have enough RAM), and are a great place to store temporary tables and other transient data.

There are a number of other features that make SQLite extremely flexible. Many of these, like virtual tables, are based off similar features found in other products, but with their own unique twist. These features and extensions provide a number of powerful tools to adapt SQLite to your own particular problem or situation.

Compatible License

SQLite, and the SQLite code, have no user license. It is not covered by the *GNU General Public License* (GPL) or any of the similar open source/free-source licenses. Rather, the SQLite development team has chosen to place the SQLite source code in the public domain. This means that they have explicitly and deliberately relinquished any claim they have to copyright or ownership of the code or derived products.

In short, this basically means you can do whatever you want with the SQLite source code, short of claiming to own it. The code and compiled libraries can be used in any way, modified in any way, redistributed in any way, and sold in any way. There are no restrictions, and no requirements or obligations to make third-party changes or modifications available to the project or the public.

The SQLite team takes this decision very seriously. Great care is taken to avoid any potential software patents or patented algorithms. All contributions to the SQLite source require a formal copyright release. Commercial contributions also require signed affidavits stating that the authors (and, if applicable, their employers) release their work into the public domain.

All of this effort is taken to ensure that integrating SQLite into your product carries along minimal legal baggage or liability, making it a viable option for almost any development effort.

Highly Reliable

The purpose of a database is to keep your data safe and organized. The SQLite development team is aware that nobody will use a database product that has a reputation for being buggy or unreliable. To maintain a high level of reliability, the core SQLite library is aggressively tested before each release.

In full, the standard SQLite test suites consists of over 10 million unit tests and query tests. The “soak test,” done prior to each release, consists of over 2.5 billion tests. The suite provides 100% statement coverage and 100% branch coverage, including edge-case errors, such as out-of-memory and out-of-storage conditions. The test suite is designed to push the system to its specified limits and beyond, providing extensive coverage of both code and operating parameters.

This high level of testing keeps the SQLite bug count relatively low. No software is perfect, but bugs that contribute to actual data-loss or database corruption are fairly rare. Most bugs that escape are performance related, where the database will do the right thing, but in the wrong way, leading to longer run-times.

Strong testing also keeps backwards compatibility extremely solid. The SQLite team takes backwards compatibility very seriously. File formats, SQL syntax, and programming APIs and behaviors have an extremely strong history of backwards compatibility. Updating to a new version of SQLite rarely causes compatibility problems.

In addition to keeping the core library reliable, the extensive testing also frees the SQLite developers to be more experimental. Whole subsystems of the SQLite code can be (and have been) ripped out and updated or replaced, with little concern about compatibility or functional differences—as long as all the tests pass. This allows the team to make significant changes with relatively little risk, constantly pushing the product and performance forward.

Like so many other aspects of the SQLite design, fewer bugs means fewer problems and less to worry about. As much as any complex piece of software can, it just works.

Uses of SQLite

SQLite is remarkably flexible in both where it can be run and how it can be used. This chapter will take a brief look at some of the roles SQLite is designed to fill. Some of these roles are similar to those taken by traditional client/server RDBMS products. Other roles take advantage of SQLite's size and ease of use, offering solutions you might not consider with a full client/server database.

Database Junior

Years of experience has taught developers that large client/server RDBMS platforms are powerful tools for safely storing, organizing, and manipulating data. Unfortunately, most large RDBMS products are resource-intensive and require a lot of upkeep. This boosts their performance and capacity, but it also limits how and where they can be practically deployed.

SQLite is designed to fill in those gaps, providing the same powerful and familiar tools for safely storing, organizing, and manipulating data in smaller, more resource constrained environments. SQLite is designed to complement, rather than replace, larger RDBMS platforms in situations where simplicity and ease of use are more important than capacity and concurrency.

This complimentary role enables applications and tools to embrace relational data management (and the years of experience that come with it), even if they're running on smaller platforms without administrative oversight. Developers may laugh at the idea of installing MySQL on a desktop system (or mobile phone!) just to support an address book application, but with SQLite this not only becomes possible, it becomes entirely practical.

Application Files

Modern desktop applications typically deal with a significant number of files. Most applications and utilities have one or more preference files. There may also be system-wide and per-user configuration files, caches, and other data that must be tracked and stored. Document-based applications also need to store and access the actual document files.

Using the SQLite library as an abstract storage layer has many advantages. A fair amount of application metadata, such as caches, state data, and configuration data, fit well with the relational data model. This makes it relatively easy to create an appropriate database design that maps cleanly and easily into an application's internal data structures.

In many cases, SQLite can also work well as a document file format. Rather than creating a custom document format, an application can simply use individual database instances to represent working documents. SQLite supports many standard datatypes, including Unicode text, as well as arbitrary binary data fields that can store images or other raw data.

Even if an application does not have particularly strong relational requirements, there are still significant advantages to using the SQLite library as a storage container. The SQLite library provides incremental updates that make it quick and easy to save small changes. The transaction system protects all file I/O against process termination and power disruption, nearly eliminating the possibility of file corruption. SQLite even provides its own file caching layer, so that very large files can be opened and processed in a limited memory footprint, without any additional work on the part of the application.

SQLite database files are cross-platform, allowing easy migration. File contents can be easily and safely shared with other applications without worrying about detailed file format specifications. The common file format also makes it easy for automated scripts or troubleshooting utilities to access the files. Multiple applications can even access the same file simultaneously, and the library will transparently take care of all required file locking and cache synchronization.

The use of SQLite can also make debugging and troubleshooting much easier, as files can be inspected and manipulated with standard database tools. You can even use standard tools to inspect and modify a database file as your application is using it. Similarly, test files can be programmatically generated outside the application, which is useful for automatic testing suites.

Using an entire database instance as a document container may sound a bit unusual, but it is worth considering. The advantages are significant and should help a developer stay focused on the core of their application, rather than worrying about file formats, caching, or data synchronization.

Application Cache

SQLite is capable of creating databases that are held entirely in memory. This is extremely useful for creating small, temporary databases that require no permanent storage.

In-memory databases are often used to cache results pulled from a more traditional RDBMS server. An application may pull a subset of data from the remote database, place it into a temporary database, and then process multiple detailed searches and refinements against the local copy. This is particularly useful when processing type-ahead suggestions, or any other interactive element that requires very quick response times.

Temporary databases can also be used to index and store nearly any type of inter-linked, cross-referenced data. Rather than designing a set of complex runtime data structures which might include multiple hash tables, trees, and cross-referenced pointers, the developer can simply design an appropriate database schema and load the data into the database.

While it might seem odd to execute SQL statements in order to extract data from an in-memory data structure, it is surprisingly efficient and can reduce development time and complexity. A database also provides an upgrade path, making it trivial to grow the data beyond the available memory or persist the data across application runs, simply by migrating to an on-disk database.

Archives and Data Stores

SQLite makes it very easy to package complex data sets into a single, easy-to-access, fully cross-platform file. Having all the data in a single file makes it much easier to distribute or download large, multi-table data stores, such as large dictionaries or geo-location references.

Unlike many RDBMS products, the SQLite library is able to access read-only database files. This allows data stores to be read directly from an optical disc or other read-only filesystem. This is especially useful for systems with limited hard drive space, such as video game consoles.

Client/Server Stand-in

SQLite works well as a “stand-in” database for those situations when a more robust RDBMS would normally be the right choice, were it available. SQLite can be especially useful for the demonstration and evaluation of applications and tools that normally depend on a database.

Consider a data analysis product that is designed to pull data from a relational database to generate reports and graphs. It can be difficult to offer downloads and evaluation

copies of such software. Even if a download is available, the software must be configured and authorized to connect to a database that contains applicable data. This presents a significant barrier for a potential customer.

Now consider an evaluation download that includes support for a bundled SQLite demonstration database. By simply downloading and running the software, customers can interact and experiment with the sample database. This makes the barrier of entry significantly lower, allowing a customer to go from downloading to running data in just a few seconds.

Similar concerns apply to traditional sales and marketing demonstrations. Reliable network connectivity is often unavailable when doing on-site demonstrations to potential clients, so it is standard practice to run a local database server for demonstration purposes. Running a local database server consumes significant resources and adds administrative overhead. Database licenses may also be a concern.

The use of SQLite eliminates these issues. The database becomes a background piece, allowing the demonstration to focus on the product. There are no database administration concerns. The simple file format also makes it easy to prepare customer-specific data sets or show off product features that significantly modify the database. All this can be done by simply making a copy of the database file before proceeding.

Beyond evaluations and demonstrations, SQLite support can be used to promote a “lite” or “personal edition” of a larger product. Adding an entry-level product that is more suitable and cost-effective for smaller installations can open up a significant number of new customers by providing a low-cost, no-fuss introduction to the product line.

SQLite support can even help with development and testing. SQLite databases are small and compact, allowing them to be attached to bug reports. They also provide an easy way to test a wide variety of situations, allowing a product to be tested against hundreds, if not thousands, of unique database instances. Even if a customer never sees an SQLite database, the integration time may easily pay for itself with improved testing and debugging capabilities.

Teaching Tool

For the student looking to learn SQL and the basics of the relational model, SQLite provides an extremely accessible environment that is easy to set up, easy to use, and easy to share. SQLite offers a full-fledged relational system that supports nearly all of the core SQL language, yet requires no server setup, no administration, and no overhead. This allows students to focus on learning SQL and data manipulation without getting bogged down by server configuration and database maintenance.

Given its compact size, it is simple to place a Windows, Mac OS X, and Linux version of the command-line tools, along with several databases, onto a small flash drive. With no installation process and fully cross-platform database files, this provides an “on the go” teaching environment that will work with nearly any computer.

The “database in a file” architecture makes it easy for students to share their work. Whole database instances can be attached to an email or posted to a discussion forum. The single-file format also makes it trivial to back up work in progress, allowing students to experiment and explore different solutions without concern over losing data.

Generic SQL Engine

SQLite virtual tables allow a developer to define the contents of a table through code. By defining a set of callback functions that fetch and return rows and columns, a developer can create a link between the SQLite data processing engine and any data source. This allows SQLite to run queries against the data source without importing the data into a standard table.

Virtual tables are an extremely useful way to generate reports or allow ad hoc queries against logs or any tabular data set. Rather than writing a set of custom search or reporting tools, the data can simply be exposed to the SQLite engine. This allows reports and queries to be expressed in SQL, a language that many developers are already familiar with using. It also enables the use of generic database visualization tools and report generators.

[Chapter 10](#) shows how to build a virtual table module that provides direct access to live web server logs.

Not the Best Choice

Although SQLite has proven itself extremely flexible, there are some roles that are outside of its design goals. While SQLite may be able to perform in these areas, it might not be the best fit. If you find yourself with any of these requirements, it may be more practical to consider a more traditional client/server RDBMS product.

High Transaction Rates

SQLite is able to support moderate transaction rates, but it is not designed to support the level of concurrent access provided by many client/server RDBMS products. Many server systems are able to provide table-level or row-level locking, allowing multiple transactions to be processed in parallel without the risk of data loss.

The concurrency protection offered by SQLite depends on file locks to protect against data loss. This model allows multiple database connections to access a database at the same time, but the whole database file must be locked in an exclusive mode to make any changes. As a result, write transactions are serialized across all database connections, limiting the overall transaction rate.

Depending on the size and complexity of your updates, SQLite might be able to handle a few hundred transactions per minute from different processes or threads. If, however, you start to see performance problems, or expect higher transaction rates, a client/server system is likely to provide better transaction performance.

Extremely Large Datasets

It is not unusual to find SQLite databases that approach a dozen gigabytes or more, but there are some practical limits to the amount of data that can (or should) be stuffed into an SQLite database. Because SQLite puts everything into a single file (and thus, a single filesystem), very large data sets can stress the capability of the operating system or filesystem design. Although most modern filesystems are capable of handling files that are a terabyte or larger, that doesn't always mean they're very good at it. Many filesystems see a significant drop in performance for random access patterns if the file starts to get into multiple gigabyte ranges.

If you need to store and process several gigabytes or more of data, it might be wise to consider a more performance-oriented product.

Access Control

An SQLite database has no authentication or authorization data. Instead, SQLite depends on filesystem permissions to control access to the raw database file. This essentially limits access to one of three states: complete read/write access, read-only access, or no access at all. Write access is absolute, and allows both data modification and the ability to alter the structure of the database itself.

While the SQLite API provides a basic application-layer authorization mechanism, it is trivial to circumvent if the user has direct access to the database file. Overall, this makes SQLite unsuitable for sensitive data stores that require per-user access control.

Client/Server

SQLite is specifically designed without a network component, and is best used as a local resource. There is no native support for providing access to multiple computers over a network, making it a poor choice as a client/server database system.

Having multiple computers access an SQLite file through a shared directory is also problematic. Most networked filesystems have poor file-locking facilities. Without the ability to properly lock the file and keep updates synchronized, the database file can easily become corrupt.

This isn't to say that client/server systems can't utilize SQLite. For example, many web servers utilize SQLite. This works because all of the web server processes are running on the same machine and are all accessing the database file from local storage.

Replication

SQLite has no internal support for database replication or redundancy. Simple replication can be achieved by copying the database file, but this must be done when nothing is attempting to modify the database.

Replication systems can be built on top of the basic database API, but such systems tend to be somewhat fragile. Overall, if you're looking for real-time replication—especially at a transaction-safe level—you'll need to look at a more complex RDBMS platform.

Most of these requirements get into a realm where complexity and administrative overhead is traded for capacity and performance. This makes sense for a large client/server RDBMS platform, but it is somewhat at odds with the SQLite design goals of staying simple and maintenance free. To keep frustration to a minimum, use the right tool for the job.

Big Name Users

The SQLite website states that, “*SQLite is the most widely deployed SQL database engine in the world.*” This is a pretty bold claim, especially considering that when most people think of relational database platforms, they usually think of names like Oracle, SQL Server, and MySQL.

It is also a claim that is difficult to support with exact numbers. Because there are no license agreements or disclosure requirements, it is hard to guess just how many SQLite databases are out there. Nobody, including the SQLite development team, is fully aware of who is using SQLite, and for what purposes.

Regardless, the list of known SQLite users adds up to an impressive list. The Firefox web browser and the Thunderbird email client both use several SQLite databases to store cookies, history, preferences, and other account data. Many products from Skype, Adobe, and McAfee also utilize the SQLite engine. The SQLite library is also integrated into a number of popular scripting languages, including PHP and Python.

Apple, Inc., has heavily embraced SQLite, meaning that every iPhone, iPod touch, and iPad, plus every copy of iTunes, and many other Macintosh applications, all ship with several SQLite databases. The Symbian, Android, BlackBerry, and Palm webOS environments all provide native SQLite support, while WinCE has third-party support. Chances are, if you have a smartphone, it has a number of SQLite databases stored on it.

All of this adds up to millions, if not billions, of SQLite databases in the wild. No doubt that most of these databases only contain a few hundred kilobytes of data, but these low-profile environments are exactly where SQLite is designed to thrive.

Large client/server RDBMS platforms have shown thousands of developers the power of relational data management systems. SQLite has brought that power out of the server room to the desktops and mobile devices of the world.